

1 Customer-specific Database Contents

1.1 HYDRA SQL syntax

The system supports different database systems: Microsoft SQL Server and Oracle. These two databases do not always use the same SQL syntax and the data types required are partly different.

MPDV software therefore uses a special SQL syntax. The internal database interfaces convert the MPDV SQL syntax dynamically into the required syntax depending on the actually used database system.

With customer-specific solutions, the HYDRA SQL syntax is less important when you perform SQL queries and statements for Data Manipulation (DML) because these statements only have to work on the customer's database system. With customer-specific developments, you can therefore use the native SQL syntax of this database system.

But you must use the HYDRA SQL syntax for Data Definition (DDL) statements so that the objects created for the customer are compatible with all areas and tools of the MPDV software.

If you create objects like tables, columns, views, triggers, indices and functions, you must use the HYDRA SQL syntax. For this purpose, use an SQL client which converts the statements into the required SQL dialects.

For further information on the HYDRA SQL syntax, refer to one of the following sections.

Only the following command line tools are authorized SQL clients for the Data Definition Language (DDL) used to create and modify data objects:

- HYDRA SQL Interpreter `hysql.exe` (Windows) or `hysql.out` (Linux)
- HYDRA Script Interpreter `hydscr.exe` (Windows) or `hydscr.out` (Linux)

HYDRA SQL Interpreter can process SQL statements including DDL in an interactive prompt or it processes text files including SQL statements one after the other. The procedure is described in detail in the following sections.

MPDV uses HYDRA Script Interpreter to execute database patches, e.g. for product upgrades.

Use the defined SQL clients and respect the conventions mentioned in the following to ensure that the customer-specific data objects are compatible with all MPDV tools.



If you do not respect these rules, serious problems might be the result when you process data objects using MPDV tools, e.g. data inconsistency, loss of DB objects like views, triggers or indices, or even loss of data. This applies in particular with upcoming release upgrades, data transfers between different systems and required database reorganizations.

1.2 HYDRA SQL Interpreter hysql

The HYDRA SQL interpreter is a command line tool on the server. To launch it, you have to start the command line for the selected system number on a Windows server. On a Linux server you have to change to the required system number with hsys.scr.

Start as follows, if you want to execute text files including SQL statements:

Windows: `hysql -r statements.sql`

Linux: `hysql.out -r statements.sql`

Start as interactive SQL command line interpreter:

Windows: `hysql -r -`

Linux: `hysql.out -r -`

The parameter "-r" returns a result row after each SQL statement. This result row contains the SQL code, the number of affected data records and in case of SELECT statements the first selected row with columns separated by the pipe character "|".

The single minus sign as last parameter starts the interactive mode.

Find examples in the sections that follow.

1.3 Namespaces for customer-specific database objects

We have defined a separate namespace for customer-specific objects in order to avoid conflicts between MPDV standard objects (e.g. new features) and customer-specific objects. MPDV standard objects do not use the customer-specific namespace.

- Prefix all customer-specific objects in the database (tables, columns, views, indices,...) with "u_".
- Columns in a customer-specific table need not start with "u_" , as the table itself is located in the customer's namespace.
- Customer-specific columns are not allowed for the standard table. Better create a customer-specific table. Maintain this table in parallel to the standard table and join it for SELECT statements. If you cannot avoid to insert customer-specific columns in standard tables, the columns must have the prefix "u_".
- If necessary, MPDV also uses the customer-specific namespace for customizations to customer-specific tables. These tables are listed in the customer documentation.

1.4 Conventions for names in the DB (tables, columns, ...)

- Names for tables, columns, etc. can be made up of the Latin letters "a" to "z", the underscore "_" and the numbers "0" to "9". The name must start with a letter. Using special characters, umlauts or characters from other character sets is not allowed.
- In the HYDRA SQL syntax, write all identifiers (table name, column name, indices, views,...) in **lower case letters**.
- The identifiers' length should not exceed 30 characters so all MPDV tools can process the objects properly.**
- Do not use other data types** than the data types specified. If you use data types that are not implemented in the MPDV SQL clients, the system will probably react with undefined and incorrect behavior.
- Use the **customer's namespace ("u_")**.

1.5 Supported data types

1.5.1 Overview

The supported data types are in some kind the "lowest common denominator". These data types are accepted in all supported database systems and implemented in all MPDV database clients.

The following table shows the data types used in HYDRA SQL and how these data types are implemented in the databases:

HYDRA SQL	Comment	Oracle	Sql Server
smallint	Integer ranging between -32768 and 32767. The value -32768 stands for an empty column (null).	NUMBER(37)	SMALLINT
integer	Integer ranging between -2147483648 und 2147483648. If the value is -2147483647, the column is empty (null).	NUMBER(22)	INTEGER
serial	See below	NUMBER(36)	INTEGER IDENTITY
decimal(m,n)	Decimal(18,6) is used by default.	NUMBER(m,n)	DECIMAL(m,n)
smallfloat	This data type is rarely used and should be avoided.	FLOAT(125)	smallfloat (→ REAL, see below)
float		FLOAT	FLOAT
float(n)	This data type is not used in the standard. Avoid this data type.	FLOAT(n)	FLOAT(n)

char or char(1)		CHAR (1)	CHAR (1)
char(n)		NVARCHAR2(n) with n > 4000 LONG	NVARCHAR(n) with n > 4000 TEXT
text	Storing large text fields. This data type is not used in the standard. Avoid this data type.	NCLOB	TEXT
date	Date without time	DATE	hydate (→ DATETIME, see below)
datetime	Timestamp including date and time.	TIMESTAMP(3)	DATETIME
image	Storing binary objects	BLOB	IMAGE

With a Microsoft SQL server, the following data types are created as user-defined data types in order to distinguish them:



hydate: EXEC sp_addtype date, datetime, 'NULL'

smallfloat: EXEC sp_addtype smallfloat, real, 'NULL'

The system automatically creates the user-defined data types when you create the database during the default installation process of the system.

1.5.2 Data type SERIAL

The data type SERIAL contains a numeric key that the database automatically assigns in ascending order. You use this data type, if a table does not contain a unique key.

ORACLE implements this data type by creating a sequence named *S_<tablename>*. This sequence provides the unique values. In addition, a trigger named *T_<tablename>* is created. If a data record is inserted into the table, this trigger reads the next value from the sequence and adds the value to the relevant column. The same trigger also guarantees that the value of the SERIAL column cannot be changed.



When you create a table including a SERIAL column, you must create a UNIQUE INDEX for this column.

1.6 Creating functions and triggers

To create functions and triggers, you also use an own syntax in HYDRA SQL for Oracle and Microsoft SQL Server because the native syntax can be quite different. MPDV therefore always lists the two statements for both SQL Server and Oracle in the corresponding SQL or HYDRA script files for functions and triggers of the standard. The clients HYDRA SQL Interpreter or HYDRA Script Interpreter execute only the relevant statement depending on the actually used database system.

You must query functions as qualified identifier preceded by database user "mipadm" (or "hydadm" preceding MW 4.0 pe) .

See also the following example:

1.7 Example

In the following example, a customer-specific table is created that includes columns with the most important data types. An index is created for the table. In addition, a function, a trigger and a view are created.

Further examples show how to write SQL statements for hysql to insert or change data in a table. The example shows how to write values for columns including data types like date, datetime and float or decimal in the statements.

1.7.1 Creating database objects

An SQL file is created in the subdirectory "db_sql" on the server:

```
create table u_machine_detail
(
  machine_nbr char(20),
  detail_text char(100),
  room_nbr integer,
  purchase_price float,
  last_maintenance_date date,
  last_maintenance_time integer,
  last_maintenance_ts datetime,
  modified_by char(10),
  modified_ts datetime,
  internal_id serial not null
);
revoke all on u_machine_detail from "public";

create unique index u_mdet_m on u_machine_detail (machine_nbr);
create unique index u_mdet_id on u_machine_detail (internal_id);

define function u_get_timestamp for oracle as
create function u_get_timestamp( p_d in date, p_n in number) return timestamp as
begin
  return cast((cast(p_d as timestamp) +
    case when (MONTHS BETWEEN(cast(p_d as timestamp),sysdate)/12)>2000 and p_n=86400 then 0
      else p_n/86400 end) as timestamp) <EOS>
end;

define function u_get_timestamp for sqlserver as
CREATE FUNCTION u_get_timestamp( @p_d hydate, @p_n int ) RETURNS datetime AS
BEGIN
  RETURN @p_d + case when @p_d=convert(datetime, 2958463) and @p_n=86400 then 0 else ((@p_n + 0.001)/86400.0) end <EOS>
END;

define trigger u_t_machine_detail_mt_ts for oracle as
CREATE TRIGGER u_t_machine_detail_mt_ts BEFORE INSERT OR UPDATE ON u_machine_detail FOR EACH ROW
BEGIN
  :new.last_maintenance_ts := get_datetime(:new.last_maintenance_date, :new.last_maintenance_time)<EOS>
END;

define trigger u_t_machine_detail_mt_ts for sqlserver as
CREATE TRIGGER u_t_machine_detail_mt_ts ON u_machine_detail FOR INSERT, UPDATE AS IF (@@ROWCOUNT = 0) RETURN
BEGIN
  SET NOCOUNT ON
  UPDATE u_machine_detail
  SET last_maintenance_ts =
    u_machine_detail.last_maintenance_date + ((u_machine_detail.last_maintenance_time + 0.001)/86400.0)
  FROM inserted
  WHERE u_machine_detail.internal_id = inserted.internal_id
  SET NOCOUNT OFF
END;

create view u_v_machine_detail_only_ts as
select machine_nbr,
  detail_text,
  room_nbr,
  purchase_price,
  hydadm.u_get_timestamp( last_maintenance_date, last_maintenance_time ) as mt_ts,
  modified_by,
  modified_ts,
  internal_id
  from u_machine_detail;
```

The SQL file is then started from the command line for the relevant system using the HYDRA SQL Interpreter hysql. The following example shows the query from a Windows server. The grayed out sections are keyboard inputs. With a Linux server, the command is "hysql.out -r db_sql/u_table_example.sql".

```

hydadm:3:F:\hydra3>hysql -r db_sql\u_table_example.sql

01.03.2017 08:56:39 PROCESSING db_sql\u_table_example.sql...

create table u_machine_detail ( machine_nbr char(20), detail_text char(100),
    room_nbr integer, purchase_price float, last_maintenance_date date, las
t_maintenance_time integer, last_maintenance_ts datetime, modified_by char(1
0), modified_ts datetime, internal_id serial not null );
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

revoke all on u_machine_detail from "public";
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

create unique index u_mdet_m on u_machine_detail (machine_nbr);
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

create unique index u_mdet_id on u_machine_detail (internal_id);
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|

define function u_get_timestamp for oracle as create function u_get_timestamp( p
_d in date, p_n in number) return timestamp as begin return cast((cast(
p_d as timestamp) + case when (MONTHS_BETWEEN(cast(p_d as timestamp),s
ysdate)/12)>2000 and p_n=86400 then 0 else p_n/86400 end) as time
stamp) <EOS> end;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

define function u_get_timestamp for sqlserver as CREATE FUNCTION u_get_timestamp
( @p_d hydate, @p_n int ) RETURNS datetime AS BEGIN RETURN @p_d + case when @p
_d=convert(datetime, 2958463) and @p_n=86400 then 0 else ((@p_n + 0.001)/86400.0
) end <EOS> END;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

define trigger u_t_machine_detail_mt_ts for oracle as CREATE TRIGGER u_t_machine
_detail_mt_ts BEFORE INSERT OR UPDATE ON u_machine_detail FOR EACH ROW BEGI
N :new.last_maintenance_ts := get_datetime(:new.last_maintenance_date, :new.l
ast_maintenance_time)<EOS> END;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

define trigger u_t_machine_detail_mt_ts for sqlserver as CREATE TRIGGER u_t_mach
ine_detail_mt_ts ON u_machine_detail FOR INSERT, UPDATE AS IF (@@ROWCOUNT = 0)
RETURN BEGIN SET NOCOUNT ON UPDATE u_machine_detail SET las
t_maintenance_ts = u_machine_detail.last_maintenance_date + ((u_machin
e_detail.last_maintenance_time + 0.001)/86400.0) FROM inserted WHERE u
_machine_detail.internal_id = inserted.internal_id SET NOCOUNT OFF END;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|-1|

```



```
create view u_v_machine_detail_only_ts as select machine_nbr, detail_text, room_nbr, purchase_price, hydadm.u_get_timestamp( last_maintenance_date, last_maintenance_time ) as mt_ts, modified_by, modified_ts, internal_id from u_machine_detail;
OK. NR OF ROWS 0.
RESULT:
|0|0|0|0|
hydadm:3:F:\hydra3>
```

1.7.2 Inserting data in a table

You can also start the HYDRA SQL Interpreter in the interactive input mode. This is useful with small SQL statements or with statements inserted via clipboard. The example also shows the output of result data with SELECT statements. Start the interactive mode using the command "hysql -r -" (Windows) or "hysql.out -r -" (Linux). (Note: the minus sign at the end!) End the interactive mode using the command "exit;".

The grayed out sections are keyboard inputs.

```
hydadm:3:F:\hydra3>hysql -r -
02.03.2017 08:40:23 PROCESSING STDIN...
SQL> insert into u_machine_detail
(
  machine_nbr,
  detail_text,
  room_nbr,
  purchase_price,
  last_maintenance_date,
  last_maintenance_time,
  modified_by,
  modified_ts
)
values
(
  '00000100',
  'Detail text for machine 00000100',
  1020,
  125000.000,
  '12/31/2016',
  14.5*3600,
  '12345',
  '12/31/2016 14:25:17.123'
);
insert into u_machine_detail ( machine_nbr, detail_text, room_nbr, purchase_price, last_maintenance_date, last_maintenance_time, modified_by, modified_ts ) values ( '00000100', 'Detail text for machine 00000100', 1020, 125000.000, '12/31/2016', 14.5*3600, '12345', '12/31/2016 14:25:17.123' );
OK. NR OF ROWS 1.
RESULT:
|0|5|1|0|
SQL>
```

The example shows:

- You can write strings in single or in double quotes.
- The dot is the decimal separator for float and decimal.
- You store time of the type *Integer* on the database tables. The content refers to "seconds since midnight". You must therefore multiply time by 3600. In the example above, the time is 14:30. Instead of entering "14.5*3600", you can directly enter 52200.
- Dates are in the 'MM/DD/YYYY' format.
- Timestamps for columns of the type datetime are in the 'MM/DD/YYYY hh:mm:ss.ccc' format. 'ccc' are milliseconds.
- A continuous number is automatically assigned to columns of the type serial, if data is inserted. The columns are not indicated in the statement.
- The created trigger automatically assigns a timestamp to the column last_maintenance_ts which is calculated from last_maintenance_date and last_maintenance_time.

1.7.3 Changing data in a table

The grayed out sections are keyboard inputs.

```
hydadm:3:F:\hydra3>mysql -r -

02.03.2017 08:40:23 PROCESSING STDIN...

SQL> update u_machine_detail set
  detail_text = 'Test machine for training',
  room_nbr = 1021,
  purchase_price = 27235.50,
  last_maintenance_date = '02/28/2017',
  last_maintenance_time = 21600,
  modified_by = 'trainee',
  modified_ts = '03/02/2017 08:54:30.468'
where machine_nbr = '00000100';
update u_machine_detail set detail_text = 'Test machine for training', room_
nbr = 1021, purchase_price = 27235.50, last_maintenance_date = '02/28/2017',
last_maintenance_time = 21600, modified_by = 'trainee', modified_ts = '03
/02/2017 08:54:30.468' where machine_nbr = '00000100';
OK. NR OF ROWS 1.
RESULT:
|0|0|1|0|

SQL>
```

Also on changing data, the created trigger automatically assigns a timestamp to the column last_maintenance_ts which is calculated from last_maintenance_date and last_maintenance_time.

You cannot change columns of the type serial. The columns can only be used as key in the WHERE clause.

1.7.4 Selecting data from a table

The grayed out sections are keyboard inputs.

```
hydadm:3:F:\hydra3>hysql -r -

01.03.2017 09:04:07 PROCESSING STDIN...

SQL> select personalnummer, name, kostenstelle, eintritt from personen where personalnummer =
40256;
select personalnummer, name, kostenstelle, eintritt from personen where personal
nummer = 40256;
OK. NR OF ROWS 1.
RESULT:
|0|0|1|0|2|4|40256|0|83|Pernikova, Lisa|0|10|105|7|10|10/09/1993|

SQL> exit;
EXIT FOUND.

hydadm:3:F:\hydra3>
```

Example for an SQL command including an error:

```
hydadm:3:F:\hydra3>hysql -r -

01.03.2017 09:11:53 PROCESSING STDIN...

SQL> select error from error where error = 'TRUE';
select error from error where error = 'TRUE';
^
ERROR -208, CISAM 0, OFFSET 0: [42S02][208][Microsoft][ODBC SQL Server Dri
ver][SQL Server]Unknown object name 'error'.
RESULT:
|-208|0|0|0|

SQL> exit;
EXIT FOUND.

hydadm:3:F:\hydra3>
```

The database system provides the error message text (e.g. MS SQL server or Oracle) and is not within the control of MPDV.

1.8 HYDRA SQL syntax reference

1.8.1 Maximum length of SQL statements

The maximum length of SQL statements is 32000 characters. Longer statements are automatically cut.
Note: necessary implementations for individual databases can increase the length.

1.8.2 Name of database objects

Do not use the following names for tables because these names have a special meaning under ORACLE:
evt_*, sm*_s, sm*_x, sm*links, smc*, smp_*. In addition, the following names are not allowed for views:
sm*_v, smp_*

1.8.2.1 Data type IMAGE

The system support the data type IMAGE (SQL server: IMAGE, Oracle: BLOB) for the following application cases:

- Creation of customer-specific tables via hysql that include data type BLOB
- Export and import of tables including BLOB data

Requirements:

- Oracle: as of MW 3.0
- SQL Server: as of MW 3.0, hyaccsql.dll version 8.1.2.10

Create tables with column type IMAGE

Requirements

Tables including one or more IMAGE columns must have a SERIAL column named **VERWEIS** (= "reference") (because of Oracle database).

Implementation

Use the tool hysql.out|exe if you want to create tables including the HYDRA data type IMAGE. The IMAGE column has data type IMAGE without any indication of size.

Example:

```
create table u_document
(
    verweis serial not null,
    ...
    opticalfingerprint image,
    ...
);
revoke all on u_document from "public";

create unique index u_document_vw on u_document (verweis);
```

Export

Store the schema of tables including IMAGE columns in the export SQL file as described in the example of section 3.1.

For each IMAGE field (per row/per column), an own IMAGE file is stored in the subdirectory <table name>, if the database field contains an IMAGE information (size > 0). The reference to the relevant file is stored in the UNLOAD file <TABLE NAME>.UNL.

All IMAGE files of a table are stored in the sub directory <TABLE NAME>.

Use the following rule to name IMAGE files:

<TABLE NAME>/<COLUMN NAME>_<current number per table>.IMG

Notes:

- Separator with WINDOWS systems is the backslash "\".
- The "current number per table" starts at 1 and is issued with 10 digits and leading zeros.
- The files have the extension: **IMG**
- All database fields in the relevant UNL file are populated with the references to the possible IMAGE files.
- Only IMAGE files that contain IMAGE information are exported to IMAGE fields.

Export example (hyexport.exe blob):

File **blob.sql**:

```
...
create table u_document
(
    verweis serial not null,
    charge_id char(20),
    documentorderid char(20),
    run char(12),
    sex char(1),
    countrycode char(3),
    nationalitycode char(3),
    documentserial char(9),
    dateofbirth datetime,
    dateofissue datetime,
    dateofexpiry datetime,
    faceimage image,
    signatureimage image,
    destinationoffice char(3),
    surname char(40),
    secondsurname char(40),
    givennames char(35),
    countrynationality char(12),
    nationality char(22),
    placeofbirth char(22),
    opticalfingerprint image,
    applicationnumber integer,
    civilbirthregistry char(30),
    diasbilityreg char(1),
    serialnumber char(10),
    residencetype char(10),
    visa char(20),
    profession char(30),
    mainfingerprtmin image,
    secfingerprtminu image,
    facialrecpattern image,
    authority char(50)
);
revoke all on u_document from "public";

create index u_document_cid on u_document (charge_id);
create unique index u_document_vw on u_document (verweis);
...
```

File **u_document.unl**

```
$COLUMN$VERWEIS|...|FACEIMAGE|SIGNATUREIMAGE|...|
24055|...|faceimage_0000000001.img|signatureimage_0000000002.img|...|
24056|...|faceimage_0000000003.img|signatureimage_0000000004.img|...|
.....
```

Storage of the **IMAGE** files

```

<HYDRADIR>
  → blob.exp [directory]
    → u_document [directory]
      faceimage_0000000001.img
      faceimage_0000000003.img
      signatureimage 0000000002.img
      ...

```

Import

If you import tables including IMAGE columns, create the tables as described in section 3.1. Perform the internal processing as described in the following:

Oracle:

- On reading the UNL files, all columns are read that are not of database type IMAGE and inserted via "INSERT INTO <TABLE>".
- The IMAGE columns are initialized using the ORACLE function **EMPTY_BLOB()**.
- Each IMAGE file (if existing) is read and inserted into the respective column.

SQL Server:

- Each IMAGE file is read before the "INSERT INTO <TABLE>" and is directly bound to the SQL statement.

1.8.2.2 Data type TEXT

The columns of HYDRA data type **char** and a length of more than 1999 characters are stored under ORACLE as data type NCLOB. The HYDRA data type is **TEXT**. You can store data up to 4Gbytes in this data type.

Restrictions / Conditions

This data type does not support the following SQL operations:

SQL Operations Not Supported	Example of unsupported usage
SELECT DISTINCT	SELECT DISTINCT clobCol from...
SELECT clause ORDER BY	SELECT... ORDER BY clobCol
SELECT clause GROUP BY	SELECT avg(num) FROM... GROUP BY clobCol
UNION, INTERSECT, MINUS (Note that UNION ALL works for LOBs.)	SELECT clobCol1 from tab1 UNION SELECT clobCol2 from tab2;
Join queries	SELECT... FROM... WHERE tab1.clobCol = tab2.clobCol
Index columns	CREATE INDEX clobIndx ON tab(clobCol)...

1.8.3 Strings

1.8.3.1 Constant strings

Use single or double quotes to limit constant strings in HYDRA SQL. ORACLE and SQL Server automatically replace double quotes by single quotes.

1.8.3.2 Substrings

Substrings with SELECT

Following the example of the previously available database INFORMIX, you can select string parts with HYDRA SQL using square brackets. Separate the first and the last digit by comma within the square brackets. ORACLE and SQL Server use the functions SUBSTR or SUBSTRING to which the first digit and the length of the result string are transferred.

```
HYDRA SQL:  select column_name[3,4] from table_name;
ORACLE:     select substr( column_name,3,2) from table_name;
SQL Server: select substring(column_name,3,2) from table_name;
```

Substrings with UPDATE

Also with UPDATE, you can access string parts with HYDRA SQL. The automatic implementation with ORACLE and SQL Server is as follows:

```
HYDRA SQL:  update table_name set column_name[3,4] = "ab";
ORACLE:     update table_name set column_name =
            substr(column_name, 1, 2 ) || 'ab' || substr(column_name, 5);
SQL Server: update table_name set column_name =
            substring(column_name, 1, 2) + 'ab' + substring(column_name, 5, 2000);
```

With SQL Server, you must always specify a length with the function SUBSTRING. But as on implementing the statement the field length is not known, a length of 2000 is assumed to add the rest of the field.

1.8.3.3 Concatenation of strings

You concatenate strings in HYDRA SQL using the operator '||'. With the SQL Server, this operator is automatically replaced with '+':

```
HYDRA SQL:  select "string" || column || "string" from tablename;
SQL Server: select 'string' + column + 'string' from tablename;
```

1.8.4 Transactions

Transactions are explicitly started with HYDRA SQL. ORACLE automatically starts a new transaction once a transaction has been finished using *commit*. If there is no active transaction, ORACLE automatically performs a *commit* after each data change:

HYDRA SQL: `begin work;`

SQL Server: `begin transaction;`

and

HYDRA SQL: `commit [work];`

SQL Server: `commit transaction;`

and

HYDRA SQL: `rollback [work];`

SQL Server: `rollback transaction;`

In case of an SQL server with default settings, a session processes a data record in a transaction and another session must wait until the transaction is finished before it has **read** access. And vice versa, **also the read access (open cursor) locks the data and the data cannot be updated by another user.**

If you use the command "**set transaction isolation level read uncommitted**" with SQL server, the second session does not wait but reads the changed (possibly not consistent) data that has not yet been "committed". This is different with ORACLE. Here, the (consistent) data is issued before the beginning of the transaction. **You nevertheless use this isolation level with SQL Server as otherwise there might be time delays and deadlocks.**

SQL Server processes nested transactions. As this is not possible with ORACLE, HYDRA SQL declines a *begin work* in a current transaction and creates an SQL error.

1.8.5 Current date, current time

HYDRA SQL uses *today* and *current* to query the current date and the current time. SQL Server offers the function *getdate()*, which returns date and time just like *sysdate* with ORACLE. When implementing *today*, the time must be cut (set to midnight) as otherwise in case of queries similar to "... where datum between today and today + 1" the current day is not included in the selection.

HYDRA SQL	ORACLE	SQL SERVER
today	trunc(sysdate,'DD')	cast(convert(char,getdate(),101) as datetime)
current	systimestamp	getdate()

Note:

Both functions always provide date and time in relation to the time of the operating system.

If you use the HYTIMEZONE functionality, the values are therefore not correct. For this reason, you may not use these two functions in the standard.

1.8.6 Date format

HYDRA SQL uses the date format "MM/TT/YYYY".

1.8.7 Date functions

The following table includes the available date functions and their implementation for ORACLE and SQL Server.

HYDRA SQL	ORACLE	SQL SERVER
year(...)	to_number(to_char(to_date(...),'YYYY'))	year(...)
month(...)	to_number(to_char(to_date(...),'MM'))	month(...)
day(...)	to_number(to_char(to_date(...),'DD'))	day(...)
weekday(...)	to_number(to_char(to_date(...),'D'))	datepart(dw,...) – 1
date(...)	to_date(...)	cast(... as datetime)
get_date(...)	trunc(...,'DD')	cast(convert(char,..., 101) as datetime)
get_time(...)	to_number(to_char(...,'SSSSS'))	(datepart(Hh,...) * 60 + datepart(Mi,...) * 60 + datepart(Ss,...)

The function get_date(...) returns the column date of type *DATETIME*. The function get_time(...) returns the column time in seconds since midnight of type *DATETIME*. Other functions

1.8.8 Other functions

Implement other functions for ORACLE and SQL Server as follows:

HYDRA SQL	ORACLE	SQL SERVER
length(...)	(no implementation)	len(...)
nvl(...)	(no implementation required)	isnull(...)
trim(...)	rtrim(ltrim(...))	rtrim(ltrim(...))
rtrim(...)	(no implementation required)	(no implementation required)
ltrim(...)	(no implementation required)	(no implementation required)
string(...)	to_char(...)	ltrim(str(...))
value(...)	to_number(...)	cast(... as integer)

The functions **trim(...)**, **rtrim(...)** and **ltrim(...)** currently only work independently of databases if a space character is used. The transfer of another character, which should be cut off on the left and/or right of a string, is not supported.

When sorting is concerned, the function **nvl(...)** has a special meaning in HYDRA SQL (see chapter "Sorting of NULL values").

1.8.9 Query of NULL values

SQL Server makes a difference between character strings including an empty string and the ones with a NULL value. At the same time, SQL Server saves the NULL values as empty string after export and subsequent reimport. Therefore, you must always combine these two queries:

HYDRA SQL: ... where (*column* is null **or** *column* = "")

With fields of type *char(1)*, you must additionally include a space character in the query:

HYDRA SQL: ... where (*column* is null **or** *column* = "" **or** *column* = " ")

The same applies, if you query not NULL:

HYDRA SQL: ... where (*column* is not null **and** *column* != "")

or

HYDRA SQL: ... where (*column* is not null **and** *column* != "" **and** *column* != " ")

1.8.10 Sort by NULL values

Contrary to SQL Server where NULL values are returned on top of the column on sorting, ORACLE sorts the NULL values at the bottom. Use the function *nvl()* to change this:

HYDRA SQL: ... order by **nvl(char_column, " ")**, **nvl(number_column, -1)**;

To enable this change with ORACLE, enter a space character (not an empty string!) as replacement value for columns of type *char*. The example above states -1 with numeric columns. If necessary, you must select a lower value. With SQL Server, this function is therefore deleted in the statement (only in *order by*).

1.8.11 Sorting with union select

If you use *union select* in a statement for sorting, you can use either the column number, the column name or the alias in the *order by* clause.

We recommend to create and use an alias with columns of a *union* that you want to sort.

1.8.12 Group by calculated expressions

If you want to group by a calculated expression, ORACLE and SQL Server expect the calculated expression in the *group by* clause. To integrate this conveniently in HYDRA SQL and to be independent of databases in the future, HYDRA SQL uses the alias to state calculated expressions in *group by*:

HYDRA SQL:	<code>select <i>column</i> + 2 <i>alias</i> from <i>table</i> group by <i>alias</i>;</code>	ORACLE +
SQL Server:	<code>select <i>column</i> + 2 <i>alias</i> from <i>table</i> group by <i>column</i> + 2;</code>	

1.8.13 Outer Join

Use the ANSI syntax for outer joins. All databases can process this syntax and for this reason it need not be implemented:

HYDRA SQL: `... from table1 a left outer join table2 b on a.column = b.column`

If you use an outer join to read in another table the name of a value of the first table, you can use a so-called lookup:

`select column1, (select column2 from table2 b where a.column1 = b.column1) from table1 a ...`

A lookup is a subselect, which replaces a column. You **must** ensure with this subselect that at least 1 data record is found, otherwise an SQL error is created. If no data record matches the condition, a NULL value is returned (similar to an outer join).

To provide compatibility, an outdated independent HYDRA SQL syntax is supported, which combines the outdated syntaxes of INFORMIX and older Oracle versions:

HYDRA SQL:	<code>... from <i>table1</i> a, outer <i>table2</i> b where <i>a.column</i> = <i>b.column</i> (+)</code>
ORACLE:	<code>... from <i>table1</i> a, <i>table2</i> b where <i>a.column</i> = <i>b.column</i> (+)</code>
SQL Server:	<code>... from <i>table1</i> a, <i>table2</i> b where <i>a.column</i> *= <i>b.column</i></code>
[INFORMIX:	<code>... from <i>table1</i> a, outer <i>table2</i> b where <i>a.column</i> = <i>b.column</i>]</code>

1.8.14 Temporary tables

1.8.14.1 Overview

Temporary tables are tables that are only valid for the current database connection and include an intermediate result. The tables are automatically deleted when the program is finished, if not yet done. The following two sections show 2 possibilities how to create a temporary table.

1.8.14.2 Restrictions

For technical reasons, the number of temporary tables is limited to 12 for one database connection.

Also programs running as services or daemons have this limit; for example the "hymw" services of the data collection. This means: When an input dialog is processed, a maximum of 12 temporary tables can be created and used at a time. If you do not explicitly drop these temporary tables, these tables decrease the number of temporary tables available in the program until the service or daemon is stopped. In this case, for example in user exits of the data collection, it is absolutely necessary to drop the temporary tables when they are not required any more to avoid problems with other software parts that also require temporary tables.

If more than 12 (or almost 12) temporary tables are required at the same time, it is more secure to use normal permanent tables. If you use permanent tables, the tables must either be dynamic tables with table names that are unique in the entire system or the data records included must be clearly linked to the database connection. To create a unique table name, you can add the terminal number to the table name or add it as additional column in the data records, for example.

1.8.14.3 create temp table

Create a temporary table using the command *create temp table*:

HYDRA SQL: `create temp table tablename (...);`

ORACLE: `create global temporary table tablename_<pid> (...)
on commit preserve rows;`

SQL Server: `create table #tablename (...);`

With ORACLE, the *global temporary tables* are available for all users. Add the PID (process ID) to the table name separated by an underscore so that the relevant process can distinctly identify the table.

1.8.14.4 select into temp

The second possibility to create a temporary table is a *select into temp*:

HYDRA SQL: `select ... from ... where ... into temp tablename;`
 ORACLE: `create global temporary table tablename_<pid>
on commit preserve rows as select ... from ... where ...;`
 SQL Server: `select ... into #tablename from ... where ...;`

1.8.15 unique / distinct

„*select unique*“ and „*select distinct*“ both work with HYDRA SQL. With SQL Server, *unique* is replaced in the statement by *distinct*.

1.8.16 like / matches

You can use *matches* as synonym of *like* in HYDRA SQL. With both comparison operators, '*' and '%' are processed as wildcard for any number of characters. '?' and '_' substitute any other character.

1.8.17 Loading and unloading data

Use the command *unload* to unload data from a database into a file. The columns of a data record are written in a row of the file separated by '|'. The command *xunload* additionally writes the row names in the first row. Use this command, if the number or the order of the columns do not coincide with those in the table into which you want to load the data. Example:

HYDRA SQL: `xunload to filename select columns from table;`

Use the command *load* to load such data back into the database. Use the command *fload* to load a great number of data records faster with ORACLE if the table includes a column of data type *serial*. To this end, delete the trigger and the sequence to create distinct values and recreate them after having loaded the data. Also use the command *fload* if the data to be loaded include very high values in the *serial* column because if you insert these high values, the sequence must be counted up to this value.

HYDRA SQL: `load from filename insert into table;`

Unload files may include comments with additional information. This way, you might store the table schema and the indices into the unload file. If you want to store comments, the following rules apply:

- You can identify the beginning of a comment because the row begins and ends with the character '\$'. Comment rows are therefore different to data rows which always end with the character '|'.

Each comment ends with "\$END\$" in a the separate row. Comment that only include one row are not possible.

The comments may include any information.

All comments must be at the beginning of the file and precede the row "\$COLUMNS\$".

Example:

```

$SCHEMA$
create table tablename ( column integer );
create unique index indexname on tablename ( column );
$END$
$VERSIONS$
hymw 8.1.1.417
$END$
$DBPATCHES$
dbp mw30 18.12.2010
$END$
$COLUMNS$column|
0|
1|

```

1.8.18 create table as select

With ORACLE and SQL Server, you can create a table from a query:

HYDRA SQL: **create table** *tablename* **as** select ... from ...;

SQL Server: select ... **into** *tablename* from ...;

1.8.19 CASE in the select clause

In the SQL-92 standard, the CASE function is implemented in SQL. Using the CASE function, you can make decisions on result set level.

Example:

In MDE log records ("ereignis" table), yield and scrap quantities may only be taken from the end-of-shift records. However, the duration of statuses has to be determined from the "N" and "P" records. Therefore, our previous programs have cumulated the quantities in a separate UNION. This problem can be solved by an SQL statement using CASE syntax. This provides fundamental performance benefits.

```

select masch_nr,
       sum(dauer),
       sum(case when (satzart = 'N') then zaehler1 else 0 end) gut,
       sum(case when (satzart = 'N') then zaehler3 else 0 end) aus
from ereignis
where masch_nr like '%'
      and bmktonr between 1 and 11
      and satzart in ('P', 'N')
group by masch_nr;

```

The following example returns name and first name separated by comma but only if a first name exists in the HR master.

```

select case when (person_vorname is null) or (person_vorname = '')
       then person_name
       else person_name || ', ' || person_vorname
       end
from personalstamm;

```

1.24 Integer division

ORACLE returns a float if you perform a division of two columns including integer data types. With SQL Server, the result and the two operands are integers:

HYDRA SQL: `select 215 / 10 from setup;`

ORACLE: `Result = 21.5`

SQL Server: `Result = 21`

You can avoid this difference by replacing one of the two numbers by a decimal value:

HYDRA SQL: `select 215 / 10.0 from setup;`

1.8.20 Changing tables

1.8.20.1 Adding columns

The different databases use different syntaxes to add columns:

HYDRA SQL: `alter table tablename add ( column1 integer, column2 char(1) );`

SQL Server: `alter table tablename add column1 integer, column2 char(1);`

1.8.20.2 Changing columns

The different databases use different syntaxes to change columns:

HYDRA SQL: `alter table tablename modify ( column1 char(20), column2 char(40) );`

SQL Server: `alter table tablename alter column column1 varchar(20);`

`alter table tablename alter column column2 varchar(40);`

1.8.20.3 Deleting columns

The different databases use different syntaxes to drop/delete columns:

HYDRA SQL: `alter table tablename drop ( column1 char(20), column2 char(40) );`

SQL Server: `alter table tablename drop column column1 varchar(20);`

`alter table tablename drop column column2 varchar(40);`

1.8.21 Reserved keyword "key"

The keyword "key" is reserved with SQL Server and DB2. If you select a column named "key", "key" must be set in double quotes.

HYDRA SQL: `select key from tabelle;`

SQL Server: `select "key" from tabelle;`

1.8.22 Default values in the database schema

If you create a table, you can define a default value for the columns.

HYDRA SQL: create table *tabelle* (*column* char(1) **default** "N" not null);
SQL Server: create table *tabelle* (*column* char(1)
 constraint df_tabelle_column default "N" not null);

Using a default value can help to avoid unnecessary *or* statements that might create errors. In addition, you can avoid NULL values in the relevant column if you add *not null*.

If you add a column with a default value to a table, this column is automatically populated in the existing data records.

HYDRA SQL: alter table *tabelle* add (*column* char(1) **default** "N" not null);
SQL Server: alter table *tabelle* add *column* char(1)
 constraint df_tabelle_column default "N" **with values** not null);

If you subsequently add a default value to an existing column, the statement must include (because of former DB Informix) the current data type. You cannot change the data type and the default value at the same time (because of SQL Server).

HYDRA SQL: alter table *tabelle* modify (*column* char(1) **default** "N" [not null]);
SQL Server: alter table *tabelle* **add constraint df_tabelle_column default** "N"
 for column;

Note:

With ORACLE, if you change a column that already is "not null", you must not state "not null" because this would create SQL error 1442.

You can delete the default value of a column with the following statement. Note that you must first reset the attribute *not null* for the column:

HYDRA SQL: alter table *tabelle* modify (*column* char(1) **default null**);
SQL Server: alter table *tabelle* **drop constraint df_tabelle_column**;

Notes:

You may only include one column per statement if you add and delete default values.

If you want to assign a default value to several columns of a table, you must use several individual SQL statements.

If you want to change the default value of a column, you must first delete the default value and then add the new default value. Here, you need not reset the attribute *not null*.

If you want to delete a column including a default value, you must first delete the default value and then drop the column (with SQL Server).

1.8.23 Process "clustered index"

With SQL Server, you can create a "clustered index" per table. With ORACLE, the keyword "clustered" is deleted from the statement:

HYDRA SQL: create [unique] **clustered** index *indexname* on ...;

ORACLE: create [unique] index *indexname* on ...;

1.8.24 Optimizing "update statistics" under ORACLE

Function

As of the below mentioned program version of the ORACLE backend, you can control the functioning of the command "update statistics" via environment variables. Using these variables, you can change from COMPUTE to ESTIMATE processing and vice versa. COMPUTE uses all data records of a table or an index to generate the statistics. ESTIMATE only uses parts of the data records (depending on the configuration).

Configuration

You control the activation of the extended functionality via environment variables.

UPD_STAT_NUM_ROWS ... Once this number of entries in a table is reached, it is changed to ESTIMATE.

UPD_STAT_ESTIMATE_PERCENT ... Percentage for ESTIMATE (value range between 1 and 100)

Example for Windows

```
set UPD_STAT_NUM_ROWS=10
set UPD_STAT_ESTIMATE_PERCENT=20
```

Example for Unix

```
export UPD_STAT_NUM_ROWS=10
export UPD_STAT_ESTIMATE_PERCENT=20
```

You can set the environment variables as described above in the script `hy_env.scr` (UNIX) or under Windows as system variable or entry in the registry.

Activation and default values

UPD_STAT_NUM_ROWS	UPD_STAT_ESTIMATE_PERCENT	Executed syntax
not set	not set	Old syntax (corresponds to COMPUTE)
Value greater than or equal to 0	not set → Default: 10 (%)	New syntax (corresponds to ESTIMATE with 10 % for tables with more than UPD_STAT_NUM_ROWS data records)
not set → Default: 0	Value between 1 and 100	New syntax (corresponds to ESTIMATE with defined percentage for all tables)
Value greater than or equal to 0	Value between 1 and 100	New syntax (corresponds to ESTIMATE with defined percentage for tables with more than UPD_STAT_NUM_ROWS data records)

Note

If the number of data records in the relevant tables is smaller than **UPD_STAT_NUM_ROWS**, the "new syntax" with `estimate_percent` with **NULL** (→ corresponds to **COMPUTE**) is used.

1.9 Notes on the performance

1.9.1 Union versus union all

Use *union* to summarize data of several tables to a result set. This *union* also removes duplicate data records. To do so, the database sorts the result set by all columns. If you do not want to remove the duplicate data records with *union* or if there cannot be duplicate data records, use the syntax *union all* instead of *union*. You save a lot of time because *union all* does not sort and remove the duplicate data records.

1.9.2 Substrings in the WHERE clause

If you use substrings in the WHERE clause, you must bear in mind that ORACLE does not use an existing index for the relevant column.

ORACLE: `select ... from auftrags_bestand where auftrag_nr[1,8] = „...“;`

In this example, ORACLE does not use the index for the column `auftrag_nr` and performs a sequential scan in worst-case.

A possible change of the statement is:

```
ORACLE:      select ... from auftrags_bestand
              where auftrag_nr like „...%“
              and auftrag_nr[1,8] = „...“;
```

1.9.3 truncate table

If you want to delete all data records of a table, use *truncate table* instead of "delete from..." with HYDRA SQL. Using this command with ORACLE, the data records to be deleted are not stored in log files and processing is accelerated.

```
HYDRA SQL:  truncate table tablename;
```

1.10 Access to several databases

1.10.1 Syntax

Currently, the access to several databases is only realized for ORACLE under Linux. Open a connection to **one** additional database using the statement:

```
connect <connectstring> [user <username>] [password <password>];
```

Example: connect linux1ora user hydadm password mpdv;

Here, the parameters *user* and *password* are optional. The default value for both parameters is "hydadm". Following the example of the local database, you can override the default values using environment variables. You can use the environment variables HYDBUSER and HYDBPW to define user and password of the local database. But you must add the connect string in capital letters separated by an underscore to add an additional database (example: HYDBUSER_DEC1ORA).

Notes:

You may only open the additional database using "connect..." once the default database has been opened.

You may not use bind variables to pass <connectstring>, <username> and <password>.

To perform statements on the second database precede the statement by

```
at <connectstring> ...
```

Example: at linux1ora select projekt from setup;

Close the additional database using the statement

close database <connectstring>;

Example: close database linux1ora;

Note:

You must close the connection to the additional database before closing the default database.

1.10.2 Restrictions

The following restrictions apply if you want to access the additional database:

- You must not create, change and delete tables (mainly because of the SERIAL columns). This also applies for the use of temporary tables.
- The command fload (fast load) is not allowed.
- You may only perform *update statistics* on the local database.
- Currently, this extension is only available under Linux (under Windows you still use HOLD_CURSOR=YES).